
LAPRAK IOT

Digital Output, Digital Input **Modifikasi Sistem Sakelar**



Oleh:

NAMA : M. Kurnia Akbar

NPM : 25782101

KELAS : TRI – 2D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2025/2026**

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Mikrokontroler ESP32 merupakan sistem dalam kemasan (*System on a Chip*) berbasis prosesor 32-bit yang menawarkan kecepatan pemrosesan tinggi, efisiensi daya, serta integrasi Wi-Fi dan Bluetooth. Keunggulan ini menjadikan ESP32 sebagai standar populer dalam pengembangan proyek *embedded system* dan *Internet of Things* (IoT).

1.2 Tujuan Praktikum

- Menerapkan logika *state tracking* untuk menyimpan kondisi output (*ON/OFF*) pada ESP32.
- Membedakan perilaku sakelar *momentary* dan *latching*.
- Menerapkan teknik *software debouncing* berbasis delay untuk mencegah *flickering* akibat pantulan tomo

1.3 Alat dan bahan

- 1x ESP32 DevKit v4
- 1x Push Button
- 1x LED Merah
- 1x Resistor 10 k Ω (*Pull-down*)
- 1x Resistor 200 Ω (*Pembatas arus LED*)
- Kabel jumper secukupnya

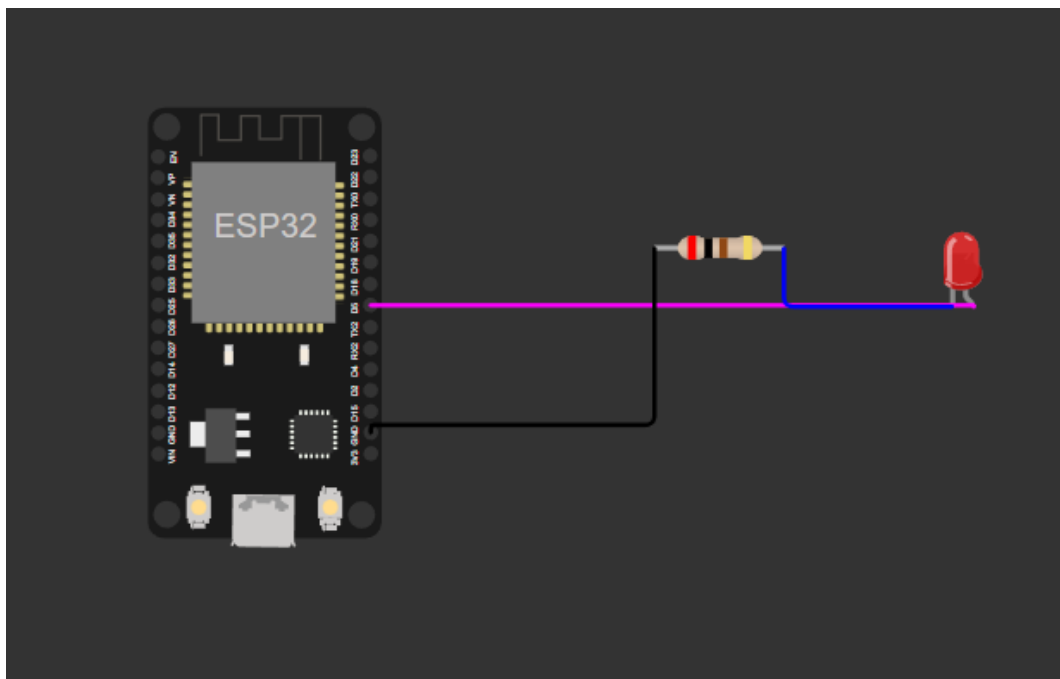
BAB II PRAKTIKUM 1

DIGITAL OUTPUT (MENYALAKAN LED)

2.1 Tujuan

Mengontrol Sinyal Keluaran Digital: Memahami konsep pemberian tegangan (*HIGH* / $3.3V$) dan pemutusan tegangan (*LOW* / $0V$) pada pin GPIO ESP32 melalui fungsi `digitalWrite()`. Sebagai Indikator Visual System: Menggunakan LED sebagai aktuator visual untuk membuktikan bahwa logika pemrosesan input dan pelacakan status (*state tracking*) pada program berhasil dieksekusi dengan benar.

2.2 Rangkaian



2.3 Langkah Kerja

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - Kaki yang lebih panjang adalah Anoda (Positif).
 - Kaki yang lebih pendek (dan terdapat sisi datar di tepi plastik kepala LED) adalah Katoda (Negatif).
3. Hubungkan pin D1 (GPIO 5) dari NodeMCU ke kaki Anoda LED menggunakan kabel *jumper*.
4. Hubungkan kaki Katoda LED ke salah satu ujung Resistor 220 Ohm.
5. Hubungkan ujung Resistor lainnya ke jalur GND pada NodeMCU. (Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar)

2.4 Kode Program

```
1  #include <Arduino.h>
2
3  const int ledPin = 5; // GPIO 5 (D5 pada ESP32 / D1 pada NodeMCU)
4
5  void setup() {
6      Serial.begin(115200);
7      pinMode(ledPin, OUTPUT);
8      Serial.println("Praktikum 1 - Digital Output Dimulai!");
9  }
10
11 void loop() {
12     digitalWrite(ledPin, HIGH);
13     Serial.println("LED Menyala");
14     delay(1000);
15
16     digitalWrite(ledPin, LOW);
17     Serial.println("LED Mati");
18     delay(1000);
19 }
```

2.5 Cara Kerja Program

Rangkaian ini menggunakan pin GPIO 5 pada ESP32 sebagai *digital output* yang terhubung ke anoda LED, serta katoda LED yang terhubung ke pin GND melalui resistor pembatas arus 200 Ω . Saat sistem berjalan, fungsi `pinMode(5, OUTPUT)` mengonfigurasi GPIO 5 sebagai keluaran, lalu perintah `digitalWrite(5, LOW)` memastikan LED berada dalam kondisi mati saat awal *booting*. Ketika program mengeksekusi `digitalWrite(5, HIGH)`, GPIO 5 mengeluarkan tegangan 3.3V yang mengalirkan arus melewati LED menuju GND sehingga LED menyala. Sebaliknya, saat perintah `digitalWrite(5, LOW)` dieksekusi, tegangan diputus menjadi 0V sehingga arus berhenti mengalir dan LED padam.

2.6 Hasil

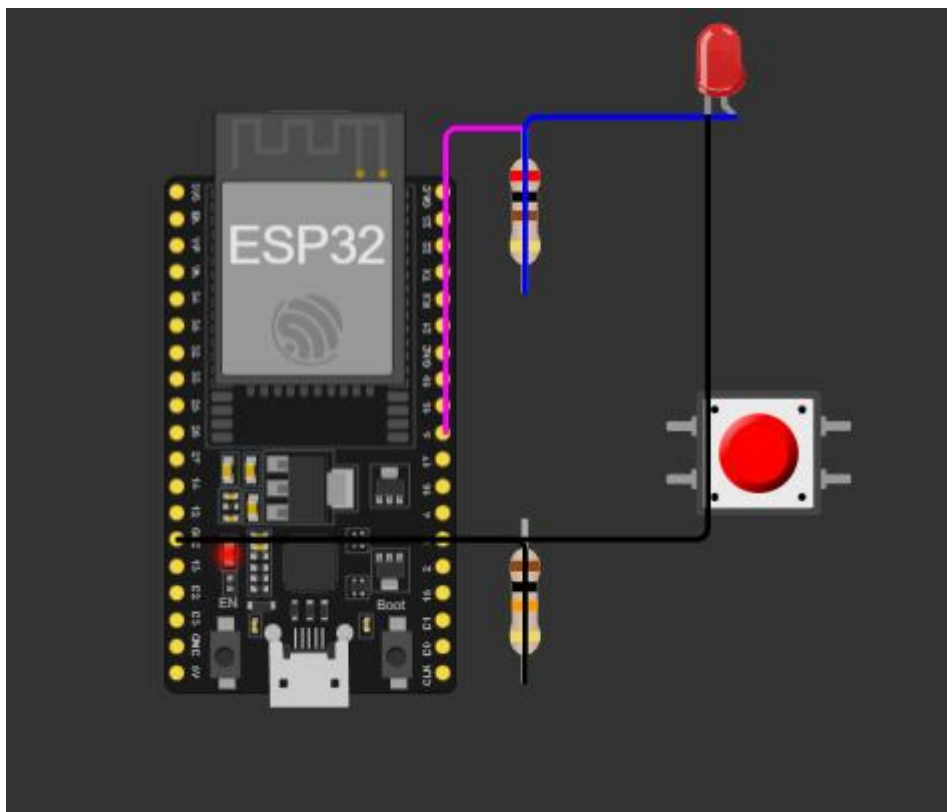
Pengujian *digital output* dan sakelar *toggle* pada ESP32 di Wokwi menunjukkan LED awal dalam keadaan mati (GPIO 5 bernilai LOW). Saat tombol ditekan 1 kali, mikrokontroler mendeteksi *rising edge* di GPIO 4 yang mengubah status variabel `ledState` menjadi `true`, membuat GPIO 5 bernilai HIGH (3.3V) sehingga LED menyala terus (*latching*). Ketika tombol ditekan lagi, nilai `ledState` menjadi `false`, GPIO 5 kembali LOW (0V), dan LED padam. Penerapan `delay(200)` juga berhasil meredam pantulan mekanis tombol sehingga mencegah efek berkedip (*flickering*).

BAB III PRAKTIKUM 2 – DIGITAL INPUT (MEMBACA PUSH BUTTON)

3.1 Tujuan

Tujuan dari praktikum ini adalah untuk memahami pembacaan sinyal *digital input* dari *push button* pada pin GPIO ESP32 menggunakan fungsi `digitalRead()`, menerapkan skema *pull-down* berbasis resistor eksternal guna menjaga kepastian status logika *LOW (0V)* saat tombol tidak ditekan, serta mengintegrasikan sinyal masukan tersebut untuk mengendalikan respon *digital output* LED secara presisi.

3.2 Rangkaian



3.3 Langkah Kerja

1. Biarkan rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan kaki pertama *push button* ke pin 3V3 (3.3V) pada NodeMCU.
4. Hubungkan kaki kedua *push button* ke pin D2 (GPIO 4) NodeMCU.
5. Pada kaki kedua yang sama, pasang sebuah Resistor 10k Ohm, lalu hubungkan ujung lain dari resistor tersebut ke jalur GND NodeMCU.

3.4 Kode Program

```
1  #include <Arduino.h>
2
3  // Deklarasi Pin (GPIO 4 = D2 NodeMCU, GPIO 5 = D1 NodeMCU)
4  const int buttonPin = 4;
5  const int ledPin = 5;
6
7  // 1. STATE TRACKING: Variabel memori untuk menyimpan status LED dan Tombol
8  bool ledState = false;    // Menyimpan kondisi LED (false = MATI, true = MENYALA)
9  int lastButtonState = LOW; // Menyimpan kondisi pembacaan tombol sebelumnya
10
11 void setup() {
12     Serial.begin(115200);
13
14     // Konfigurasi pin input (Rangkaian Pull-Down Eksternal 10k Ohm)
15     pinMode(buttonPin, INPUT);
16     pinMode(ledPin, OUTPUT);
17
18     // Kriteria 1: Memastikan LED dalam keadaan MATI saat pertama kali dinyalakan
19     digitalWrite(ledPin, LOW);
20 }
21
22 void loop() {
23     // Membaca status logika tombol saat ini
24     int currentButtonState = digitalRead(buttonPin);
25
26     // LOGIKA STATE TRACKING & RISING EDGE DETECTION:
27     // Periksa hanya saat terjadi perubahan dari LOW ke HIGH (saat tombol BARU DITEKAN)
28     if (currentButtonState == HIGH && lastButtonState == LOW) {
29
30         // Toggle Status: Membalikkan nilai boolean (True -> False, False -> True)
31         ledState = !ledState;
32
33         // Memperbarui kondisi fisik LED sesuai nilai ledState
34         digitalWrite(ledPin, ledState ? HIGH : LOW);
35
36         // Menampilkan status pada Serial Monitor
37         if (ledState) {
38             Serial.println("Tombol Ditekan 1x -> LED MENYALA (Latching ON)");
39         } else {
40             Serial.println("Tombol Ditekan 1x -> LED MATI (Latching OFF)");
41         }
42
43         // 2. DEBOUNCING: Memberikan jeda 200 ms untuk meredam pantulan mekanis tombol fisik
44         // Mencegah efek flickering (perubahan status acak/berkedip saat ditekan)
45         delay(200);
46     }
47
48     // Menyimpan pembacaan saat ini untuk dibandingkan pada iterasi loop berikutnya
49     lastButtonState = currentButtonState;
50 }
```

3.5 Cara Kerja Program

Program bekerja dengan membaca sinyal masukan dari *push button* pada pin GPIO 4 menggunakan fungsi `digitalRead()` dalam rangkaian *pull-down* (bernilai LOW saat dilepas dan HIGH saat ditekan). Ketika tombol ditekan dan terdeteksi transisi sinyal dari LOW ke HIGH (*rising edge*), program akan membalikkan status variabel boolean `ledState` (`!ledState`) serta mengirimkan perintah `digitalWrite()` ke pin GPIO 5 untuk mengubah kondisi LED (*ON/OFF*), lalu memberikan jeda `delay(200)` sebagai *debouncing* untuk meredam pantulan mekanis tombol.

3.6 Hasil

Hasil pengujian menunjukkan bahwa saat tombol tidak ditekan, pin GPIO 4 membaca sinyal LOW (0V) berkat skema *pull-down*, sehingga LED di GPIO 5 tetap berada dalam kondisi mati. Ketika tombol ditekan satu kali, mikrokontroler membaca sinyal HIGH (3.3V) dan mendeteksi *rising edge* yang mengubah nilai `ledState` menjadi `true`, sehingga GPIO 5 aktif menyalakan LED secara stabil (*latching*). Saat tombol ditekan kembali, sinyal input baru membalikkan status `ledState` menjadi `false` yang mematikan LED secara permanen, sementara jeda `delay(200)` berhasil meredam pantulan mekanis (*bounce*) tombol sehingga perubahan kondisi LED berjalan presisi tanpa *flickering*.

BAB IV LATIHAN

4.1 Latihan 1 – Pemahaman Logika Pin

Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jawaban: Jika Anda menyambungkan tombol dengan logika *Active-High (pull-down)* ke pin D8 (GPIO 15) dan menekannya saat NodeMCU baru diberi daya, NodeMCU akan mengalami *boot failure* atau *stuck* saat *startup*. Hal ini terjadi karena pin D8 merupakan pin *boot strapping* khusus yang secara mutlak wajib berada dalam kondisi *LOW* (0V) saat proses *power-on* atau *reset* agar mikrokontroler ESP8266 dapat masuk ke mode *booting* dari memori *flash* internal. Ketika tombol ditekan saat daya dinyalakan, arus 3.3V dari rangkaian *pull-down* memaksa sinyal pin D8 menjadi *HIGH*, sehingga melanggar syarat fisik *startup* dan menyebabkan mikrokontroler kebingungan memilih *boot mode* hingga sistem akhirnya *freeze* sebelum program sempat dijalankan.

4.2 Latihan 2 (Implementasi Kode)

Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.

```
C++

digitalWrite(ledPin, HIGH); // LED Menyala
delay(200);                 // Nyala cepat selama 200 ms

digitalWrite(ledPin, LOW);  // LED Mati
delay(800);                 // Mati agak lama selama 800 ms
```


4.3 Latihan 3 (Analisis Sirkuit)

Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Jawaban: Jika kaki LED dipasang terbalik (Anoda ke GND dan Katoda ke D1), program tidak akan mengalami error kompilasi karena kesalahan tersebut berada pada level sirkuit fisik (*hardware*), bukan pada sintaks kode C/C++. LED sendiri merupakan dioda yang memiliki polaritas dan hanya mengizinkan arus searah mengalir dari Anoda ke Katoda (*forward bias*). Pemasangan Anoda ke GND (0V) dan Katoda ke D1 (3.3V saat HIGH) menyebabkan LED berada dalam kondisi prasikap mundur (*reverse bias*) yang menyumbat aliran arus listrik, sehingga beda potensial yang dihasilkan tidak mampu mengalirkan arus dan membuat LED tetap mati.

4.4 Latihan 4 (Pengembangan Algoritma)

Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

```
C++  
  
if (digitalRead(buttonPin) == HIGH) {  
    digitalWrite(ledPin, LOW); // Saat tombol DITEKAN, LED mati  
} else {  
    digitalWrite(ledPin, HIGH); // Saat tombol TIDAK ditekan, LED menyala  
}
```

Pada rangkaian *pull-down*, `digitalRead(buttonPin)` akan bernilai HIGH saat tombol ditekan dan LOW saat tombol dilepas. Dengan membalik perintah `digitalWrite()`, logika program diubah sehingga ketika tombol ditekan (HIGH), GPIO 5 memutus arus ke LOW untuk mematikan LED. Sebaliknya, saat tombol dilepas (LOW), GPIO 5 secara otomatis mengirimkan sinyal HIGH untuk menyalakan LED

BAB V TUGAS

MODIFIKASI SISTEM SAKELAR *TOGGLE (LATCHING)*

5.1 Tujuan

Tujuan dari modifikasi sistem sakelar *toggle (latching)* ini adalah untuk mengonversi perilaku *push button* dari sakelar sesaat (*momentary*) menjadi sakelar pengunci (*latching*), mengimplementasikan variabel penyimpan status (*state tracking*) untuk mengontrol kondisi *ON/OFF* LED secara independen, menerapkan logika deteksi transisi sinyal (*rising edge*) agar perubahan status hanya dipicu satu kali per tekanan, serta memanfaatkan teknik *software debouncing* (*delay(200)*) guna meredam pantulan mekanis tombol dan mencegah timbulnya efek *flickering* pada LED.

5.2 Rangkaian

Rangkaian ini menggunakan push button sebagai *digital input* pada pin GPIO 4 dengan skema *pull-down* eksternal melalui resistor 10 k Ω ke GND agar pin membaca sinyal LOW (0V) saat tombol dilepas dan HIGH (3.3V) saat ditekan, serta LED Merah sebagai *digital output* pada pin GPIO 5 yang terhubung secara seri dengan resistor 200 Ω sebagai pembatas arus menuju GND.

5.3 Kode Program

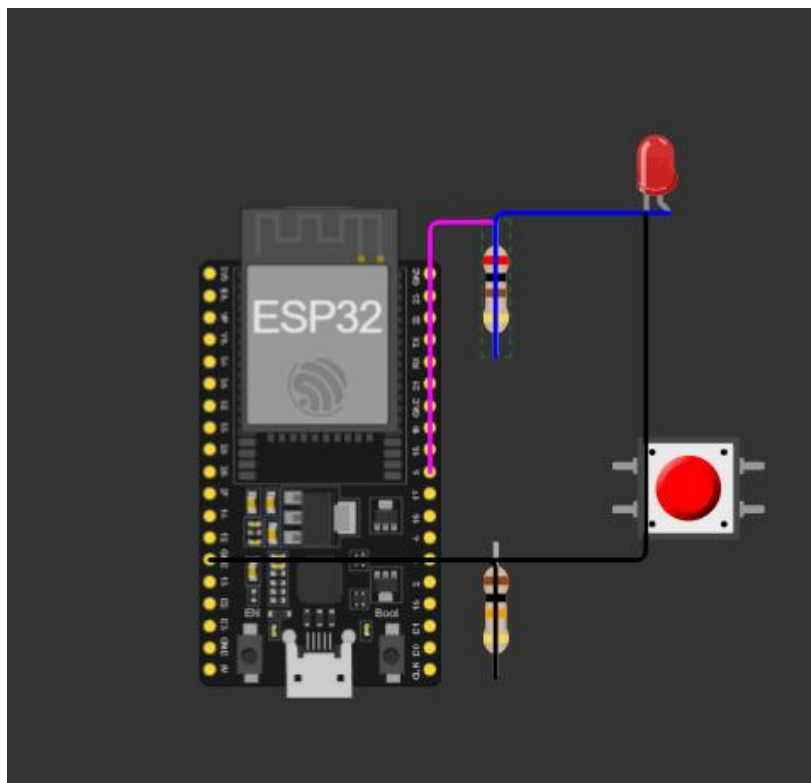
```
1  #include <Arduino.h>
2
3  // Deklarasi Pin (GPIO 4 = D2 NodeMCU, GPIO 5 = D1 NodeMCU)
4  const int buttonPin = 4;
5  const int ledPin = 5;
6
7  // 1. STATE TRACKING: Variabel memori untuk menyimpan status LED dan Tombol
8  bool ledState = false; // Menyimpan kondisi LED (false = MATI, true = MENYALA)
9  int lastButtonState = LOW; // Menyimpan kondisi pembacaan tombol sebelumnya
10
11 void setup() {
12   Serial.begin(115200);
13
14   // Konfigurasi pin input (Rangkaian Pull-Down Eksternal 10k Ohm)
15   pinMode(buttonPin, INPUT);
16   pinMode(ledPin, OUTPUT);
17
18   // Kriteria 1: Memastikan LED dalam keadaan MATI saat pertama kali dinyalakan
19   digitalWrite(ledPin, LOW);
20 }
21
22 void loop() {
23   // Membaca status logika tombol saat ini
24   int currentButtonState = digitalRead(buttonPin);
25
26   // LOGIKA STATE TRACKING & RISING EDGE DETECTION:
27   // Periksa hanya saat terjadi perubahan dari LOW ke HIGH (saat tombol BARU DITEKAN)
28   if (currentButtonState == HIGH && lastButtonState == LOW) {
29
30     // Toggle Status: Membalikkan nilai boolean (True -> False, False -> True)
31     ledState = !ledState;
32
33     // Memperbarui kondisi fisik LED sesuai nilai ledState
34     digitalWrite(ledPin, ledState ? HIGH : LOW);
35
36     // Menampilkan status pada Serial Monitor
37     if (ledState) {
38       Serial.println("Tombol Ditekan 1x -> LED MENYALA (Latching ON)");
39     } else {
40       Serial.println("Tombol Ditekan 1x -> LED MATI (Latching OFF)");
41     }
42
43     // 2. DEBOUNCING: Memberikan jeda 200 ms untuk meredam pantulan mekanis tombol fisik
44     // Mencegah efek flickering (perubahan status acak/berkedip saat ditekan)
45     delay(200);
46   }
47
48   // Menyimpan pembacaan saat ini untuk dibandingkan pada iterasi loop berikutnya
49   lastButtonState = currentButtonState;
50 }
```

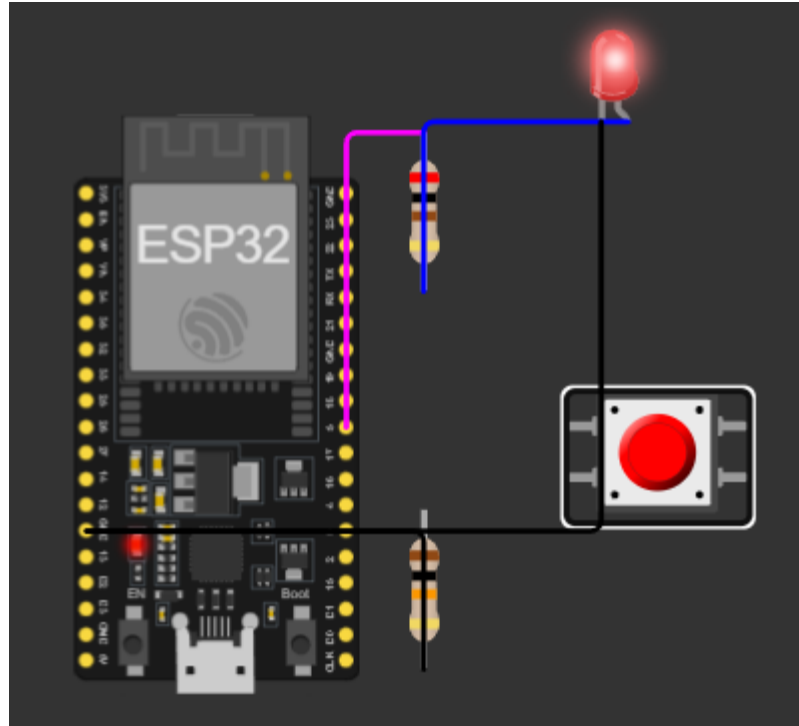
5.4 Penjelasan logika

Logika program ini bertumpu pada mekanisme *state tracking* menggunakan variabel boolean `ledState` untuk mengingat kondisi LED (*ON/OFF*) secara mandiri, yang dipadukan dengan *rising edge detection* via kondisi `if (currentButtonState == HIGH && lastButtonState == LOW)` agar pembalikan status (`ledState = !ledState`) hanya dipicu satu kali tepat saat tombol mulai ditekan. Hasil perubahan logika tersebut langsung dikirim ke GPIO 5 melalui `digitalWrite()` untuk mengubah kondisi fisik LED, sementara instruksi `delay(200)` mengeksekusi *software debouncing* untuk meredam pantulan mekanis tombol fisik guna mencegah efek *flickering* sebelum variabel `lastButtonState` diperbarui untuk siklus pembacaan berikutnya.

5.5 Hasil Pengujian

Hasil pengujian menunjukkan bahwa ketika sistem pertama kali diberi daya, LED berada dalam kondisi mati dan *Serial Monitor* belum menampilkan status karena tombol belum dipicu. Saat tombol ditekan satu kali singkat, sistem mendeteksi *rising edge* di GPIO 4 yang mengubah variabel `ledState` menjadi `true`, menyalakan LED di GPIO 5 secara stabil (*latching ON*), serta menampilkan pesan "Tombol Ditekan 1x -> LED MENYALA (Latching ON)" pada *Serial Monitor*. Ketika tombol ditekan kembali untuk kedua kalinya, sinyal masukan membalikkan `ledState` menjadi `false`, mematikan LED secara permanen (*latching OFF*), dan mencetak pesan konfirmasi "Tombol Ditekan 1x -> LED MATI (Latching OFF)". Seluruh proses perpindahan status ini berjalan presisi tanpa pemicuan ganda atau *flickering* berkat jeda *debouncing* 200 ms yang meredam pantulan mekanis tombol.





BAB VI KESIMPULAN

Berdasarkan seluruh rangkaian praktikum, latihan, dan modifikasi sistem yang telah dilakukan, dapat disimpulkan bahwa:

1. Pengendalian *Digital Output & Input*: Mikrokontroler ESP32 berhasil mengontrol aktuator LED secara presisi menggunakan perintah `digitalWrite()` pada pin GPIO 5 (3.3V untuk HIGH / 0V untuk LOW), serta membaca status sinyal digital dari *push button* secara stabil menggunakan `digitalRead()` pada pin GPIO 4 yang dilengkapi skema *pull-down* eksternal (resistor 10 k Ω) untuk menghindari kondisi *floating*.
2. Implementasi *State Tracking & Latching*: Konversi perilaku *push button* dari sakelar sesaat (*momentary*) menjadi sakelar pengunci (*latching*) sukses diterapkan memanfaatkan kombinasi variabel penanda status (`ledState`) dan deteksi transisi sinyal *rising edge* (`currentButtonState == HIGH && lastButtonState == LOW`). Mekanisme ini memastikan perubahan status LED hanya dipicu satu kali per satu kali tekanan tombol secara independen.
3. Optimasi Sistem & Pemahaman Hardware: Teknik *software debouncing* berbasis `delay(200)` terbukti efektif meredam pantulan mekanis (*bounce*) tombol sehingga mencegah timbulnya pemicuan ganda (*flickering*). Selain itu, pemahaman aspek sirkuit fisik sangat krusial—termasuk memperhatikan polaritas LED untuk menghindari kondisi *reverse bias*, serta menghindari penggunaan pin *boot strapping* seperti GPIO 15 (D8) dalam skema *active-high* agar tidak memicu kegagalan *booting* (*stuck*) pada mikrokontroler.